

Can Language Models Pretend Solvers? Logic Code Simulation with LLMs

Minyu Chen[1], Guoqiang Li[1(✉)], Ling-I Wu[1], Ruibang Liu[1], Yuxin Su[1], Xi Chang[2], and Jianxin Xue[2]

[1] Shanghai Jiao Tong University, Shanghai 200240, China
{minkow,li.g,edithwuly,628628,sshirley}@sjtu.edu.cn
[2] Shanghai Polytechnic University, Shanghai 201209, China
{changxi,jxxue}@sspu.edu.cn

Abstract. Logical solvers are typically employed for the static analysis and formal verification of functional code that is not amenable to verification through execution. Concurrently, some research efforts aim to convert natural language into logic code to bolster reasoning capabilities with the aid of logical solvers. However, the transformation process from code or natural language to a logical framework is costly. To address this challenge, several studies have explored the use of large language models (LLMs) to facilitate this transformation. Nevertheless, LLMs may introduce misalignment and formatting errors that are unacceptable for logical solvers. Considering the strong logical reasoning capabilities of LLMs and their relative insensitivity to alignment and formatting errors, we propose utilizing LLMs to emulate logical solvers for logic code simulation. In light of the characteristic fixed outputs of logical solvers, we introduce a novel LLM-based code simulation technique called Dual Chains of Logic (DCoL). Additionally, we have compiled three new datasets specifically for the Logic Code Simulation task and conducted comprehensive evaluations. These assessments demonstrate state-of-the-art performance when compared to other LLM prompt strategies, achieving a notable 7.06% improvement in accuracy with GPT-4 Turbo.

Keywords: Large Language Models · Logic Solvers · Code Simulation · Code Understanding · Evaluation

1 Introduction

Logic serves as the foundation for the majority of formal methodologies in software engineering. By translating specifications, constraints, and test cases into logical frameworks like First Order Logic (FOL) or Satisfiability Modulo Theories (SMT), logic solvers can precisely judge arguments in problems or check the satisfiability of problems. Thus, state-of-the-art solvers including Z3 [8] and CVC5 [1] are applied in wide aspects of software engineering, including software verification [7], software testing [31], program synthesis [15], and program

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2025
T. Bourke et al. (Eds.): SETTA 2024, LNCS 15469, pp. 102–121, 2025.
https://doi.org/10.1007/978-981-96-0602-3_6

analysis [11]. However, logic remains a complex subject within human cognition, presenting several challenges to the deployment of logic solvers in software engineering. One key obstacle is the gap between natural language (NL) and the solver language (SL) of logical problems, impeding not only the programming process but also the comprehension of code segments within existing software systems. Besides, despite advancements, solvers encounter difficulties in effectively and precisely resolving logical problems, including those that are straightforward for humans. Furthermore, the integration of logic in software engineering often involves various extensions like arrays, integers, reals, strings, and bit-vectors. This diversity has led to the development of specialized solvers [5,11] tailored to different logical scopes.

Recent advances in transformer-based large language models (LLMs) such as GPT [4] and LLaMA [35] have showcased their ability to perform logic reasoning akin to humans [19,25]. By incentivizing LLMs to translate natural language into solver languages with in-context learning (ICL) [4] or model fine-tuning [10], LLMs can successfully solve simple propositional logic questions in NL form [28,34]. When faced with intricate logic-based queries in forms such as FOL [14] and SAT [47], LLMs exhibit challenges in direct problem-solving and show low successful execution rates of code generation. LLMs also fail to conduct relational reasoning [20]. Nonetheless, it is crucial to recognize the valuable bridge LLMs establish between natural language and solver language, along with the code intelligence LLMs bring to various aspects of software engineering, such as code generation [22,44,45], documentation [23,41], code understanding [36], and others [9,13,32,33,42]. On the other hand, a novel line of research, code simulation, is raised to discover the ability of LLMs to simulate the execution of code and algorithms. Preliminary research reports that the LLMs struggle to execute long and complex procedures [18].

This research focuses on previously overlooked areas that have been left unexplored by prior studies. While previous works have predominantly focused on directly solving logic problems presented in natural language (NL) form or employing external logical solvers, Large Language Models (LLMs) have primarily served as NL reasoners or NL-to-SL translators in those endeavors. LLMs have been extensively utilized for logic reasoning in NL; however, the most frequently encountered logic problems in software engineering are found in solvers. Solvers check the logic formulas inherently implicit within code, which are manifested through the code's execution. These logic code in solvers serve as fundamental components that guide the analysis process and influence the identification of defects, verification of correctness properties, and generation of test cases. Although the logic problems derived from code differ in format from those in NL, LLMs have demonstrated sufficient capability to resolve logic problems in solvers, such as code understanding and logic problem solving. Thus, we propose an intriguing research question: Can language models simulate logic codes?

To explore this question, we introduce our framework, *Dual Chains of Logic (DCoL)*, the first framework specifically designed for LLM-based logic solvers. Given the characteristics of logic solvers, with outputs typically fixed as `True`

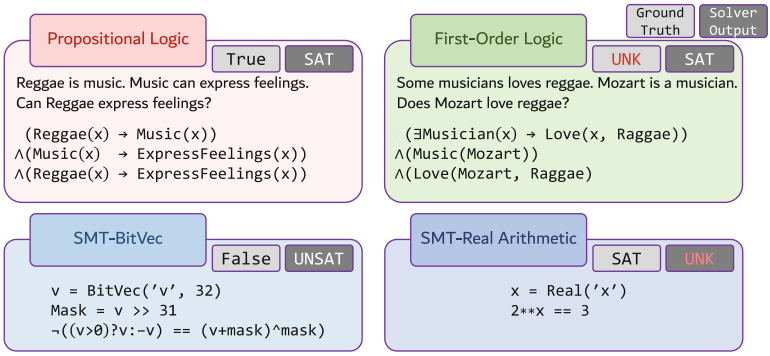


Fig. 1. Samples of logical problem to be studied in our research. The **light** gray boxes display the ground truth of the given problems, while the results given by the SMT solver are presented in the **dark** grey boxes. 'UNK' denotes unknown.

(SAT), `False` (UNSAT), or `Unknown` (UNK), DCoL directly processes code as input and guides LLMs to adopt a dual-path reasoning approach, enabling them to accurately infer the code's outcome. This framework helps LLMs effectively navigate reasoning challenges, such as handling multiple assignment problems, and reduces confusion during logical problem-solving. Through experiments, we demonstrate LLMs' competence in predicting the outcomes of codes involving logical reasoning. Additionally, we investigate the effectiveness of various prompting techniques in enhancing LLMs' performance in simulating code. Furthermore, we discuss the potential benefits that LLMs can bring to the current state of logic solvers, such as tolerance for slips in logic codes.

We highlight our contributions as follows:

- We are the **first** to propose a novel task, Logic Code Simulation, which assesses the ability of Large Language Models (LLMs) to directly reason the output of logical framework code based on their logical reasoning capabilities, unaffected by non-standard formats.
- We introduce a novel LLM-based code simulation technique, Dual Chains of Logic (DCoL), which leverages the fixed outputs characteristic of logic solvers. This technique incorporates the concept of backward chaining [16] by assuming an initial output and then guiding the LLM to determine whether contradictions exist, thereby achieving more reliable results.
- We have compiled three new datasets specifically for the Logic Code Simulation task: Z3Tutorial, Z3Test, and SMTSIM. Furthermore, we conducted comprehensive evaluations on various LLMs, demonstrating that DCoL achieves a notable 7.06% improvement in accuracy with GPT-4-Turbo.

2 Background and Related Work

Large Language Model. Large Language Models (LLMs) are generative models based on the pre-trained Transformer architecture. Most LLMs utilize a gen-

erative model architecture, where given a sentence of n tokens, the model is trained to maximize the likelihood of the ground-truth token t_i at the current time step t based on its preceding sequence $t_{i-1}, ..., t_1$. The training of LLMs typically follows three main processes: unsupervised training on large amounts of unlabeled text data without explicit human annotations, supervised fine-tuning on labeled data relevant to specific tasks or domains, and reinforcement learning on feedback from human annotators or evaluators. Leveraging extensive multimodal data and employing pre-training and fine-tuning techniques, LLMs have demonstrated state-of-the-art performance across various downstream tasks, such as machine translation, numerical reasoning, and code clone detection, with minimal examples (few-shot) or task-specific prompt instructions (zero-shot).

Logical Solver. We begin by identifying logical problems to be addressed in this work. Generally speaking, the hardest logical problem underlies the framework of satisfiability modulo theories (SMT). SMT is a variation of the SAT problem for first-order logic (FOL), with the interpretation of symbols constrained by specific theories, such as real arithmetic and bit vectors. Our study encompasses several subsets of SMT problems, including propositional logic, SMT problems themselves, and a segment of first-order logic within the solving ability of SMT, along with external theories defining constraints. Thus, we focus on SMT-based languages and solvers, to universally encode and solve those logical problems.

We elaborate on several samples to be solved in Fig. 1. In the realm of propositional logic, we incorporate variables like x to enhance conceptual clarity. The proposed hypothesis, *Reggae(x)* $\rightarrow$ *ExpressFeelings(x)*, can be derived with deductive reasoning. We transform the judgment process into a satisfiability dilemma by treating the assumption as a constraint. FOL supports quantifiers $\forall$ and $\exists$ on the basis of propositional logic, introducing uncertainty into problems. In this case, the assumption remains indeterminate. We check both the affirmation and negation hypotheses with SMT solvers, to align with the True/False/Uncertain output form. The uncertain cases are equivalent to UNSAT for both affirmation and negation hypotheses. Concerning SMT-based problems, theories also contribute to their complexity. In the SMT-BitVec case, the target is to ascertain whether the Left-Hand-Side (LHS) equals the Right-Hand-Side (RHS) for a bit-operation-oriented issue. The `True/False` corresponds to SAT/UNSAT. Conversely, the SMT-Real-Arthmetic is drawn from the Z3py manual. The Z3 solver produces an unknown result as x is involved in the exponentiation of 2^x, rendering this problem non-polynomial. In contrast, individuals can readily find a solution as $x = \log_2 3$.

Code Simulation with LLMs. Code simulation is a relatively new topic of interest. It is defined as predicting the concrete outputs of codes aligning with the execution results. Recent studies on Turing Completeness of transformers and LLMs [12,26,29,39] have suggested that LLMs are capable of interpreting instructions from code, showing potential to simulate the execution of code and algorithms as analog models. The previous works encourage LLMs to simulate a variety of tasks, such as keyboard and mouse actions on computers [17] and optimizers [42]. While transformer-based models are trained to predict the exe-

cution traces of codes [21] and UNSAT cores [30], a recent attempt at simulating code with LLMs has been proposed [18]. Their results indicate that LLMs can execute instructions sequentially.

When dealing with long programs and complex procedures, various strategies, such as memorization, can be employed. Our proposed logic code simulation framework focuses on simulating specialized codes for logical solvers rather than general-purpose code. The potential outcome space is constrained to either SAT or UNSAT, which makes accurate predictions more attainable, as logical codes can be more clearly expressed and understood in natural language.

3 DCoL: Dual Chains of Logic

In this section, we present a new prompt technique, *Dual Chains of Logic* (DCoL), aiming to improve the accuracy, reasoning process, and robustness of the logic code simulation task.

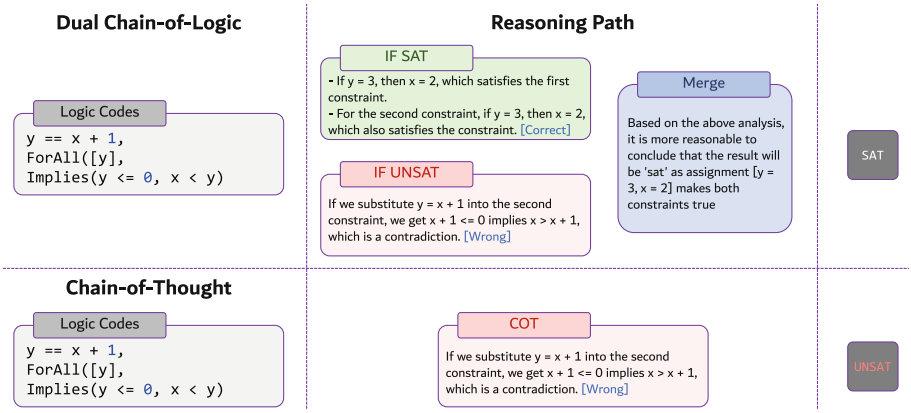


Fig. 2. Overview of the DCoL method: DCoL offers two hypotheses, SAT (satisfiable) and UNSAT (unsatisfiable), for logic code simulation. The LLMs verify these hypotheses individually before combining them to reach the final decision, while the COT method only outputs one possible reasoning path.

Since the concept of reasoning paths is crucial, they lead to more precise outcomes. Nevertheless, when dealing with general questions like mathematical computations and text generation, the answer search space is practically infinite, posing challenges in constructing reasoning paths. In contrast, logic problems that can be converted into solvers typically result in *dual* possible outcomes: SAT or UNSAT. The reasoning paths of humans for determining whether a set of constraints is SAT or UNSAT are diverse:

(1) If the result is SAT, we should find a legal assignment of variables to verify it. Consider the case presented in Fig. 3. Assuming that the given codes produce the output SAT, an arrangement of variables that fulfills all constraints

serves as compelling evidence. One possible assignment is $x = 2, y = 3$, which satisfies the first constraint $y \equiv x + 1$. In regard to the second constraint, $\forall y\ [(y \leq 0) \implies (x < y)]$, y is a **independent variable**[1], unrelated to the y defined in $y \equiv x + 1$. Therefore, only $x = 3$ is relevant in this context, and this assignment meets the second constraint.

(2) If the result is UNSAT, we should find a set of minimal constraints where conflicts always exist, which is also known as UNSAT core. Replacing $x > y$ with $x < y$ in the second constraint transforms the problem into an UNSAT one. The UNSAT core solely consists of the second constraint because, regardless of the value assigned to x, there exists a value for y that is non-positive and less than the given x value, meaning the second constraint cannot always be satisfied.

CoT Prompt

```
Suppose you are one of the greatest mathematicians, logician, and programmer. Given the following code in Z3 solver, you are asked to determine the output of the code with #sat or #unsat. Let's think step by step.

Code: [[CODE]]
```

DCoL Prompt

```
Given the following code in Z3 solver, you are asked to determine the output of the code with #sat or #unsat. Follow instructions below:

1. List and understand variables and constraints added to the solver.

2. Based on the constraints, if I say the result is 'sat', is it correct? Can you find satisfied assignments for variables? Let's think step by step.

3. Based on the constraints, if I say the result is 'unsat', is it correct? Can you find conflict constraints? Let's think step by step.

4. Which is more correct and reasonable based on the above aspects? Answer your preferred hypothesis with #sat or #unsat. Let's think step by step.

Code: [[CODE]]
```

Fig. 3. Prompting template of the DCoL method. Prompts can be modified slightly according to specific tasks.

However, LLMs cannot fully grasp the dual implicit reasoning chains for logic problems during one execution time. Among the dual paths, at least one leads to an accurate solution. Thus, by guiding LLMs to reason via both SAT/UNSAT paths, LLMs can produce different reasoning paths for contrary goals, and generate promising answers with explanations.

To this end, we propose a novel prompt method, Dual Chains of Logic (DCoL). Figure 2 illustrates our proposed DCoL approach. DCoL encourages LLMs to discard incorrect wrong prediction UNSAT after integrating dual paths

[1] Independent variable y can be replaced with any symbol, such as $\forall z\ [(z \leq 0) \implies (x < z)]$.

of thinking, while COT only generates a wrong reasoning path, leading to the prediction error. As illustrated in the template of DCoL from Fig. 3, the DCoL prompt begins by asking the user to analyze a given logic code, such as a SAT problem or Z3 solver code, to determine whether the result is #sat (satisfiable) or #unsat (unsatisfiable). It guides the user through a structured reasoning process: first, they are instructed to list and understand the variables and constraints added to the solver. Next, the prompt asks the user to verify if the result is #sat by checking if there are valid assignments for the variables. Then, the user is asked to consider if the result is #unsat and find any conflicting constraints. Finally, the user must decide which outcome—#sat or #unsat—is more reasonable based on their analysis and explain their conclusion. The logic code to be analyzed is represented by the placeholder "Code: [[CODE]]" at the bottom of the prompt, where the actual code would be inserted for evaluation.

DCoL, in contrast to COT, steers clear of falling into the erroneous reasoning trap illustrated in Fig. 2. Since y functions as an independent variable, it is inaccurate to interchangeably use $y = x + 1$ in the second constraint, given the realm that y in both constraints does not hold the same significance. This error, albeit subtle, often goes unnoticed even by humans lacking specialized knowledge. However, DCoL consistently provides a precise reasoning approach for obtaining accurate outcomes by thinking on the dual side of logic problems.

4 Experimental Results

In this section, we present the effectiveness and robustness of LLM-based logic solvers. We will provide a comprehensive analysis of the errors observed in unsuccessful results. Finally, we will discuss the performance of GPT-family-based logic solvers on challenging problems.

4.1 Involved LLMs

The objective of this study is to assess LLMs' reliability and limitations in simulating various forms of logic codes compared to conventional logical solvers like Z3. To accomplish this, we employ both open-source LLMs such as the LLaMA family [27,35], and close-source but strong LLMs such as the GPT family as our base models. We present details of these models as follows:

GPT-3.5 Turbo: GPT-3.5 Turbo, also known as ChatGPT, is a decoder-only network based on the Transformer architecture, comprising 175 billion parameters. It has undergone pretraining on 45 terabytes of text data gathered from diverse sources like books, articles, and websites. Moreover, GPT-3.5 Turbo is optimized for invoking its capability with Reinforcement Learning from Human Feedback (RLHF) technique. The GPT-3.5 Turbo-0125 supports a context window of 16,385 tokens.

GPT-4 Turbo. GPT-4 Turbo is an advancement of GPT-4, offering increased capabilities with an updated knowledge cutoff extended to April 2023. It also features a 128k context window in a single prompt.

LLaMA-2. LLaMA-2, an open-source foundational model introduced by Meta, has been trained on 2 trillion tokens and is designed to accommodate a context length of 4096 by default. LLaMA-2 models have been refined with the input of more than 1 million human annotations, specifically tailored for chat purposes. This study focuses on the evaluation of the 13B-Chat model.

Code LLaMA. Code LLaMA is a refined version of LLaMA-2 to support various code-related activities like coding, testing, explaining, and finishing code segments. We also deploy CodeLLaMA-13B-Instruct for our evaluation.

4.2 Datasets

To delve deeper into simulating real-life logical codes, we propose 3 new datasets from various sources. (1) We gathered 31 code examples from the Programming Z3 tutorial [3], culminating in a new dataset named *Z3Tutorial*. (2) Additionally, the Z3 official repository, Z3test[2], offers a range of test samples for the Z3 solver, from which we extracted 85 Python cases to form the *Z3Test* dataset. These cases were then categorized into three question types: logic-only, arithmetic, and type-inference. We discard the type-inference questions such as *is_add(x+y)* in our primary experimental setting, but will be introduced as a challenge for logic code simulation. (3) we included several codes in SMT-LIB format from the SMT-COMP 2023 [2]. These instances are predominantly derived from industrial software or established logical problem collections. We have chosen various external theories to enhance the intricacy of the problems, including real arithmetic (both linear and non-linear), bit vectors, uninterpreted functions, and strings. Our *SMTSim* dataset consists of 102 samples in total. For further insights, we present a comprehensive overview of our curated dataset in Table 1. Note that the complexity of a logic problem is determined by both the quantity of constraints and the difficulty of logic expressions. The number of lines is only a partial factor of the overall complexity. All those code snippets are ensured to only produce the result of SAT or UNSAT.

4.3 Prompting Baselines

The baseline Prompting methods considered in this work are listed below:

- Standard Prompting (SD) [4]. SD directly asks LLMs to predict the execution results of logic codes. The LLMs will return only SAT/UNSAT, sometimes with short explanations.
- Chain-of-Thought (COT) [40]. COT encourages LLMs to generate a systematic solution by explaining each step before presenting the final answer. The LLMs will return a reasoning path tackling the issue raised.
- Plan-and-Solve Prompting (PS) [37]. PS instructs LLMs to devise a detailed plan for addressing issues and solve plan targets sequentially.

[2] https://github.com/Z3Prover/z3test.

Table 1. Dataset information provided. Format means the input format of questions. Z3Py is based on Python, while SMTLIB is the standard language of SMT solvers. Mean LOC represents the average Lines of Code (LoC) of examples.

Dataset	Formulation	Format	# of Samples	Mean LoC
ProntoQA	PL	NL	500	-
Z3Tutorial	SMT	Z3Py	31	9.90
Z3Test	SMT	Z3Py	85	8.37
SMTSIM	SMT	SMTLIB	104	14.36

– Chain of Simulation (CoSm) [18]. The CoSm method enhances the conventional Chain of Thought prompting technique by circumventing the drawbacks associated with memorization in code simulation tasks. It aims to track the program trace and forces LLMs to simulate each instruction sequentially.

Additionally, Self-Consistency (SC) [38] utilizes a majority voting mechanism across different reasoning paths to enhance prediction robustness. The key insight behind this method is ensuring consistency in the reasoning process. Since SC operates as a parallel prompting method, it can be combined with other prompting approaches, including ours. Therefore, we evaluate it as a standalone plugin.

Note that our proposed task is a novel one focusing on codes as initial input. Thus, methods for processing natural language problems [25,43,46], are no longer compatible with our framework. Furthermore, these approaches are limited in their ability to address a specific portion of the task we have outlined, such as SAT [46] and SMT-Int (SMT questions with integer computing theory) [25,43]. As a result, we do not include them as baselines in our study.

A notable observation is that the typical few-shot prompt, which is also known as in-context learning is ineffective in our experimental setting, which is also reported in similar tasks such as code simulation [18] and discussed in several works [24]. Few-shot prompt learning may not work on a task whose input-label correspondence is not already captured in the LLMs. In our logic code simulation task, elementary operations in different subtasks such as different function calls and various theories can be challenging for in-context learning. Besides, considering the intricate nature of logic problems, the templates heavily exceed the LLMs' context limit. Thus, we focus on the zero-shot prompting scheme in this research.

Table 2. Performance of LLMs with different prompt techniques. The **bolded** numbers denote the best performance in this dataset, while results in grey boxes represent the best result among all the prompt methods. All experiments are conducted three times.

Method		Z3Tutorial			Z3Test		
		Accuracy	Unknown	Exe. Acc.	Accuracy	Unknown	Exe. Acc.
Z3		100.0	-	100.0	98.80	1.20	100.0
GPT-3.5 Turbo	SD	67.74	-	67.74	70.59	-	70.59
	PS	67.74	-	67.74	72.54	3.52	75.19
	CoSm	63.44	-	63.44	71.76	-	71.76
	COT	68.82	-	68.82	74.11	0.39	74.40
	DCoL	70.97		70.97	74.90	-	74.90
GPT-4 Turbo	SD	84.95	-	84.95	76.47	3.53	79.27
	PS	**86.02**	-	86.02	82.35	-	82.35
	CoSm	81.72	-	81.72	80.39	2.35	82.32
	COT	**86.02**	-	86.02	81.18	2.35	83.13
	DCoL	**86.02**	-	86.02	**83.53**	1.76	84.52
LLaMA 2-13B	SD	38.70	19.35	48.00	40.00	8.23	43.03
	PS	54.83	9.67	60.71	42.35	8.23	46.15
	CoSm	61.29	9.67	67.85	34.11	2.35	34.93
	COT	53.22	8.06	57.89	50.00	5.88	52.79
	DCoL	64.51	6.45	68.96	54.11	1.17	54.76
Code LLaMA 13B	SD	58.06	3.22	60.00	57.64	5.88	61.25
	PS	51.61	12.90	59.25	55.29	10.58	61.84
	CoSm	70.96	-	70.96	61.17	9.41	67.53
	COT	56.45	17.74	68.62	58.23	3.52	60.36
	DCoL	70.96	6.45	75.86	62.35	-	62.35

4.4 Effectiveness

We employ four LLMs, GPT-3.5-Turbo, GPT-4-Turbo, LLaMA-2-13B, and CodeLLaMA-13B, with different prompting methods, to explore their capability of them on simulating logic codes. The results are provided in Table 2. Two datasets based on Z3Py are utilized for a comprehensive analysis.

We utilize two additional metrics, namely "unknown rate" and "execution accuracy" to further investigate the effectiveness of logic code simulation.

Unknown rate indicates the percentage of instances such as 'If a solution exists, the output will be SAT; otherwise UNSAT', where LLMs struggle to provide a definitive answer. An unknown output can be caused by errors in the program, such as 'The code should output ERROR rather than SAT or UNSAT'.

Execution accuracy refers to the precision among instances that LLMs confirm to be SAT/UNSAT. This measure aids in assessing the reliability of the outputs generated by LLMs. This metric can be computed as $EA = \frac{Acc}{1-UNK}$.

A model can have low accuracy while obtaining a high execution accuracy. If we apply a random guessing procedure on those unknown examples, the accuracy will be acceptable. Note that all methods obtain execution accuracy greater than 50%, which means LLMs can perform at least stronger than random guessing.

The main observations are summarized below:

GPT Can Pretend Logic Solvers. As shown in Table 2, GPT shows its capability to simulate logic codes even with the standard prompting method (SD). In the Z3Tutorial dataset analysis, it is evident that employing standard prompts can yield an accuracy exceeding 80% when utilizing GPT-4 Turbo, known for its high resilience among LLM options. Even when employing SD prompts with GPT-3.5, an accuracy of 67.74% can be attained. As for the Z3Test dataset, GPT-3.5 achieves over 70% accuracy, while GPT-4 successfully solved 80% logic problems in this dataset. This phenomenon comes from the combination of code understanding and logical reasoning for GPT models. Moreover, among all LLMs tested, the Dual Chains of Logic (DCoL) consistently enhances performance in logic code simulation across all datasets, with gains of up to 7.06% on GPT-4.

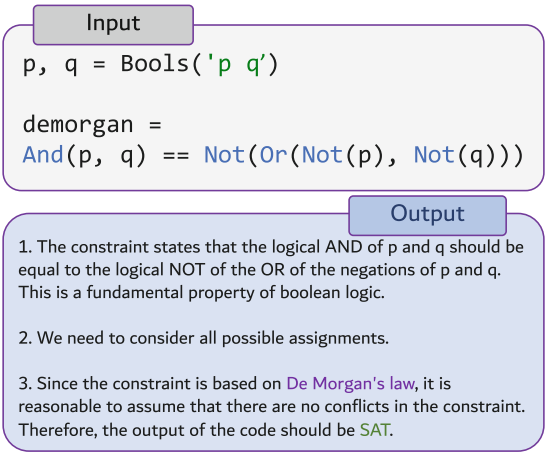


Fig. 4. An example of De Morgan's Law

Think on Logic Rather than Think on Code. A representative method performing 'Think on Code' is Chain of Simulation (CoSm), which forces LLMs to simulate the program instruction by instruction. However, this method is only suitable for simulating problems with sequential execution paths. Logic reasoning is a more complex issue as the execution chain of logic codes is hidden under external solver libraries such as the Conflict Driven Clause Learning algorithm applied in Z3 solver. To solve logic problems, LLMs should pay more attention to logical reasoning to derive the correct execution path. Nevertheless, the execution of

code should not be ignored in our task. A typical error case involves adding constraints to a solver that are never actually verified, where LLMs should disregard these constraints.

LLMs Exceed Some Theoretical Limitations of Solvers. A motivated example of this strength is showcased in Fig. 1. Constraints like $2^x == 3$ go beyond the representation capability of SMT theories. Although it remains such a simple question for humans, typical solvers like Z3 output UNKNOWN in this case.

LLMs Leverage Explainable Knowledge for Reasoning. We provide an interesting example in Fig. 4 targeting checking the satisfiability of De Morgan's Law. Despite enumerating all assignments for proving, LLMs build the connection from logic codes to De Morgan's Law and make the correct prediction with this external knowledge. We suggest that leveraging formulas and theorems is a good manual for complex problem-solving, and LLMs show their capability of utilizing knowledge for more explainable reasoning.

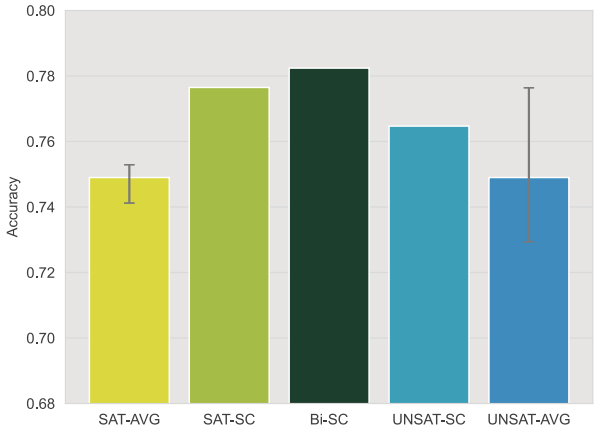


Fig. 5. Prompting template of the DCoL method. Prompts can be modified slightly according to specific tasks.

DCoL Prompt is Effective. Our proposed prompt method, Dual Chains of Logic (DCoL), offers a structured approach to LLM reasoning by encouraging two separate lines of problem-solving, and the results in Table 2 effectively support its efficacy. Across different datasets, particularly with the Z3Tutorial and Z3Test, we see that DCoL consistently outperforms other methods or shows competitive results, especially in the execution accuracy (Exe. Acc.) and accuracy rates, where it often achieves the best or near-best performance (highlighted in grey). For instance, GPT-4 Turbo achieves 86.02% accuracy using DCoL in the Z3Tutorial dataset, matching other top-performing methods while outperforming others in execution accuracy on the Z3Test dataset with 83.53%. This consistent performance boost suggests that the dual-reasoning structure introduced by DCoL is highly compatible with the evolving capabilities of LLMs. It

facilitates multi-way reasoning, allowing the model to explore multiple solution paths, which can reduce the "unknown" output rate, indicating fewer situations where the model fails to provide meaningful answers. While the GPT models may occasionally follow a single reasoning pathway, as seen in experiments, the results show that DCoL still effectively improves overall model accuracy and reduces failure cases, confirming its utility in logic code simulation and problem-solving.

Bi-directional Self-consistency Improves Performance. A recent study suggests that the sequence in which premises are given to LLMs significantly impacts their performance in natural language reasoning tasks [6]. The Dual Chains of Logic prompt we introduced will present SAT and UNSAT questions to LLMs in a successive manner. By swapping the order of SAT/UNSAT reasoning path in DCoL, LLMs can show different tending for the final prediction. Inspired by the idea of self-consistency, To explore this further, a validation experiment was designed based on the concept of self-consistency. The experiment involved sending 3 SAT-first and 3 UNSAT-first DCoL prompts to Z3Test via GPT-3.5-Turbo separately, followed by aggregating the responses using a majority voting approach. The experimental results are depicted in Fig. 5, showing that the bi-directional Self-Consistency (Bi-SC) mechanism resulted in a 3.34% improvement to the average performance by bridging the gap between execution orders.

Based on the experimental results, we believe that LLMs, particularly the GPT family, **serve as highly effective simulators**, especially when utilizing our proposed DCoL prompt and Bi-directional Self-Consistency mechanism.

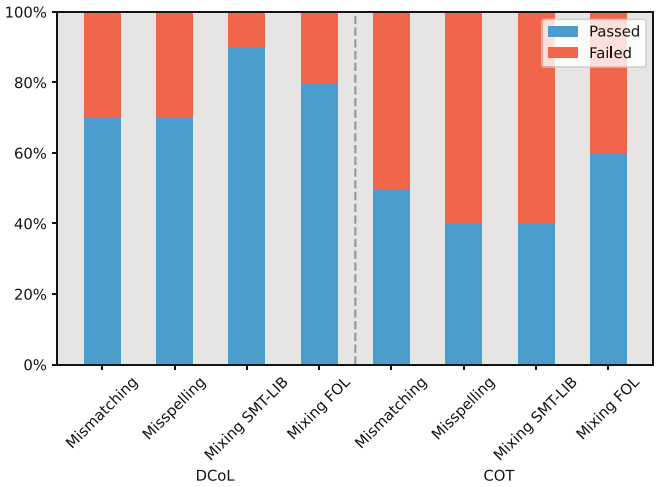


Fig. 6. Performance drop against code-level modifications

4.5 Robustness

We have already demonstrated the capability of Large Language Models (LLMs) in code simulation, indicating that LLMs can comprehend our intentions based on the provided code. Since LLMs can predict the outcome of code without actually executing it, they are more tolerant of syntax errors compared to logic solvers. To assess the robustness of code simulation in LLMs, we conducted an experiment involving four distinct syntax error patterns:

- Mismatched Parentheses: there is an inconsistency in the placement of parentheses, such as an unequal number of opening and closing parentheses or improper nesting, such as $(a + b$ or $func(a, b))$.
- Misspelled Variable or Function Names: incorrect spelling or naming of variables or functions within the code, such as $x1$ and xl.
- Mixing Z3 Library Grammar: elements specific to the SMT-LIB, such as $(declare-fun\ \langle symbol\rangle\langle sort\rangle^*)$, are incorrectly integrated into the code syntax.
- Mixing First-Order Logic Grammar: elements of first-order logic grammar, such as quantifiers $\forall$ and $\exists$, are incorrectly incorporated into the code syntax.

For each syntax error pattern, we randomly selected 10 cases from the Z3 Test dataset to introduce errors. Subsequently, we applied both COT and our proposed method (DCoT) on the poisoned Z3 Test dataset. As illustrated in Fig. 6, the code simulation of LLMs with COT was significantly misled by the introduced syntax errors, leading to a decrease in accuracy of approximately 40%. In contrast, syntax errors had minimal influence on the effectiveness of our method, which demonstrates **LLMs are robust simulators**.

Table 3. ProntoQA Comparison

Method	Accuracy
Direct Solving	
SD	51.20
COT	72.00
COT + CodeGen	76.80
Translation + Solver	
Logic-LM	61.00
Translation + Simulation	
Code Only	50.20
NL + Code	75.80

LLM Can Simulate Generated Logic Codes. This section is motivated by the limitations of Logic-LM [25], which tackles logic inference in natural language through a translation-solver approach. While Logic-LM achieves almost 100%

success in generating symbolic codes for the ProntoQA dataset, its accuracy sharply drops to 61% when interacting with symbolic solvers. This decline is largely due to **LLMs generating unreliable or implicit codes**, where hidden knowledge inferred by the LLM is not explicitly present in the generated code. For instance, an LLM might generate a code segment that assumes a certain variable should be non-negative based on common sense reasoning, even though this constraint is not included in the code itself. Consequently, a traditional symbolic solver, unaware of this implicit assumption, may infer an incorrect answer. To mitigate this issue, we propose ensuring consistency between code generation and the solver's understanding by simulating logic on generated outputs. As shown in Table 3, we conducted a comparative study based on Logic-LM's shortcomings. The *COT + CodeGen* approach treats code generation as an auxiliary in-context task, while *NL + Code* provides the original language input along with previously generated code. Our results indicate that even with low-quality code, direct simulation performs comparably to standard prompting for natural language tasks, with *NL + Code* achieving an accuracy of 75.80%, which is comparable to *COT + CodeGen* and outperforms the Logic-LM baseline that relies on external solvers.

4.6 Error Analysis

We analyzed results from the Z3Test dataset using three prompting methods: CoSm, COT, and DCoL, aiming to identify error types across different reasoning paths. The errors were categorized into six types:

Inference Error: Mistakes in logical deduction, such as misjudging $a \wedge b$ as false when both a and b are `True`, or conflicting conclusions like first inferring $a > 10$ and then $a < 10$.

Misunderstanding Satisfiability (SAT) Error: Occurs when an LLM correctly identifies a solution set for a satisfiable (SAT) problem but wrongly classifies it as unsatisfiable (UNSAT), due to misinterpreting the problem as having multiple solutions.

Partial UNSAT Error: Arises when the ground truth is satisfiable, but the LLM finds conflicting assignments while ignoring valid ones, wrongly classifying the problem as UNSAT. For example, $x = 1, y = 0$ is satisfiable, but the LLM may mistakenly label $x = 1, y = 2$ as an UNSAT core.

Bit-vector Arithmetic Error: Involves errors in handling bitwise operations on bit vectors, like incorrectly interpreting a left shift operation as an overflow.

Real Arithmetic Error: Includes computational mistakes with real numbers, such as confusion over the equality of positive and negative zeros in floating-point numbers.

Commonsense Error: Basic mathematical or logical mistakes, such as equating 1 with 2 or failing to recognize common library functions, indicating gaps in the LLM's understanding.

Our analysis (in Fig. 7) shows that DCoL improves LLMs' understanding of satisfiability problems, reducing misinterpretations and incorrect judgments.

In arithmetic tasks, DCoL enhances performance by guiding the LLM through initial steps.

While not the focus of our study, certain industry examples are noteworthy. For instance, the `is_add()` function in Z3Py, which checks if an expression is an addition, can lead to confusion; `is_add(1 + 2)` may return false as Z3Py treats $1 + 2$ as a constant, potentially confusing LLMs and users unfamiliar with this behavior.

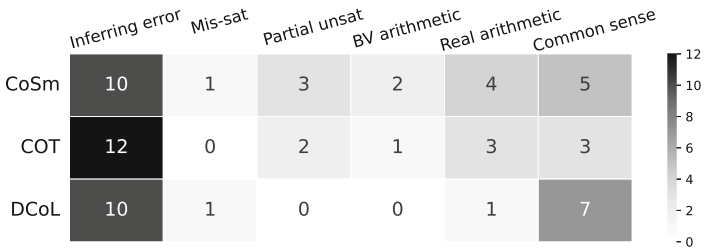


Fig. 7. Error analysis among different methods

Additionally, in the *SMTSim* dataset from SMT-COMP 2023, some SMT files generated by complex backend libraries contain numerous variables and formulas. Due to LLM token limits, we filtered out larger examples, yet many of these filtered cases remain unsolved by solvers like Z3, suggesting potential areas for future research.

4.7 Hard Problems

SMTSim dataset contains complicated logic code, with more lines of codes, variables, and theories. We regard it as a challenge for logic code simulation. We report the performance of GPT families in Table 4.

Table 4. SMTSim Results

Method		ACC.	UNK.	Exe. Acc
Z3 Solver		97.06	2.94	100.0
GPT-3.5 Turbo	COT	5.88	85.29	33.33
	CoSm	14.71	78.43	68.20
	DCoL	**54.9**	6.86	58.94
GPT-4 Turbo	COT	51.96	19.61	65.92
	CoSm	50.98	23.53	66.67
	DCoL	**58.82**	5.82	62.45

We observe that COT and CoSm suffer from UNKNOWN prediction, while our proposed DCoL achieves over 50% accuracy even with GPT-3.5-Turbo. However, the relatively low execution accuracy reveals that LLMs struggle to solve such complex datasets. This issue may stem from the abundance of variables and constraints present. In the SMTSim dataset, numerous instances occur infrequently but are deeply intertwined. Due to their limitations in handling extensive contexts, LLMs struggle to address complex logical compositions effectively.

5 Conclusion

In this work, we introduce the novel concept of logic code simulation to evaluate the ability to comprehend, analyze, and simulate logic problems encoded in programs. We propose DCoL, a novel prompt fashion designed to enhance reasoning performance on code-based logic problems. Furthermore, we release three datasets collected from the solver community and perform an extensible assessment to evaluate the capability of logic code simulation for various LLMs. In the conclusion, the authors mention a potential avenue for future work involving the application of the DCoL technique to convert natural language into logical code. Apriori, it is not immediately clear how this can be achieved, but exploring this direction could provide valuable insights.

One possible approach involves leveraging the dual reasoning paths central to DCoL, where the first path focuses on interpreting natural language inputs and extracting the logical structure or constraints embedded in the text. The second path could then focus on translating this understanding into executable logical code, ensuring consistency between the extracted logic and the generated code. This dual-path system could help bridge the gap between natural language reasoning and formal logic, addressing the ambiguity that often arises when translating nuanced human language into strict logical frameworks. Additionally, integrating techniques such as knowledge retrieval and structured prompting could allow LLMs to retrieve relevant logical patterns from a knowledge base while maintaining flexibility in the translation process.

This integration would represent a significant step forward in developing LLM-based solvers capable of handling not only traditional logic problems but also more complex, real-world scenarios where logical rules are derived from natural language descriptions. By enhancing DCoL in this way, future research could establish a new paradigm in logic-based problem-solving, where natural language inputs are seamlessly converted into formal logic programs. Furthermore, we plan to broaden DCoL's application beyond traditional solvers like SAT or UNSAT, ensuring its effectiveness across diverse logic code scenarios and complex logic programs, paving the way for more efficient LLM-driven logic solvers.

Acknowledgement. This work is supported by the National Natural Science Foundation of China Grant No. 61872232.

References

1. Barbosa, H., et al.: cvc5: a versatile and industrial-strength SMT solver. In: Fisman, D., Rosu, G. (eds.) TACAS 2022. LNCS, vol. 13243, pp. 415–442. Springer, Cham (2022). https://doi.org/10.1007/978-3-030-99524-9_24
2. Beyer, D.: Competition on software verification and witness validation: SV-COMP 2023. In: Sankaranarayanan, S., Sharygina, N. (eds.) TACAS 2023. LNCS, vol. 13994, pp. 495–522. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-30820-8_29
3. Bjørner, N., de Moura, L., Nachmanson, L., Wintersteiger, C.M.: Programming Z3. In: Bowen, J.P., Liu, Z., Zhang, Z. (eds.) SETSS 2018. LNCS, vol. 11430, pp. 148–201. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-17601-3_4
4. Brown, T., et al.: Language models are few-shot learners. In: Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901 (2020)
5. Cai, S., Li, B., Zhang, X.: Local search for SMT on linear integer arithmetic. In: Shoham, S., Vizel, Y. (eds.) CAV 2022. LNCS, vol. 13372, pp. 227–248. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-13188-2_12
6. Chen, X., Chi, R.A., Wang, X., Zhou, D.: Premise order matters in reasoning with large language models. arXiv preprint arXiv:2402.08939 (2024)
7. Cordeiro, L., Fischer, B.: Verifying multi-threaded software using SMT-based context-bounded model checking. In: Proceedings of the 33rd International Conference on Software Engineering, pp. 331–340 (2011)
8. de Moura, L., Bjørner, N.: Z3: an efficient SMT solver. In: Ramakrishnan, C.R., Rehof, J. (eds.) TACAS 2008. LNCS, vol. 4963, pp. 337–340. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-78800-3_24
9. Deng, Y., Xia, C.S., Yang, C., Zhang, S.D., Yang, S., Zhang, L.: Large language models are edge-case generators: crafting unusual programs for fuzzing deep learning libraries. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, pp. 1–13 (2024)
10. Feng, J., et al.: Language models can be logical solvers. arXiv preprint arXiv:2311.06158 (2023)
11. Gadelha, M.R., Steffinlongo, E., Cordeiro, L.C., Fischer, B., Nicole, D.: SMT-based refutation of spurious bug reports in the clang static analyzer. In: 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), pp. 11–14. IEEE (2019)
12. Giannou, A., Rajput, S., Sohn, J., Lee, K., Lee, J.D., Papailiopoulos, D.: Looped transformers as programmable computers. In: International Conference on Machine Learning, pp. 11398–11442. PMLR (2023)
13. Gupta, P., et al.: Grace: language models meet code edits. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 1483–1495 (2023)
14. Han, S., et al.: Folio: natural language reasoning with first-order logic. arXiv preprint arXiv:2209.00840 (2022)
15. Kang, E., Lafortune, S., Tripakis, S.: Automated synthesis of secure platform mappings. In: Dillig, I., Tasiran, S. (eds.) CAV 2019. LNCS, vol. 11561, pp. 219–237. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-25540-4_12
16. Kazemi, M., Kim, N., Bhatia, D., Xu, X., Ramachandran, D.: Lambada: backward chaining for automated reasoning in natural language. arXiv preprint arXiv:2212.13894 (2022)

17. Kim, G., Baldi, P., McAleer, S.: Language models can solve computer tasks. In: Advances in Neural Information Processing Systems, vol. 36 (2024)
18. La Malfa, E., et al.: Code simulation challenges for large language models. arXiv preprint arXiv:2401.09074 (2024)
19. Lee, J., Hwang, W.: Symba: symbolic backward chaining for multi-step natural language reasoning. arXiv preprint arXiv:2402.12806 (2024)
20. Li, Z., et al.: LLMs for relational reasoning: How far are we? arXiv preprint arXiv:2401.09042 (2024)
21. Liu, C., et al.: Code execution with pre-trained language models. In: Findings of the Association for Computational Linguistics: ACL 2023, pp. 4984–4999 (2023)
22. Liu, M., Yang, T., Lou, Y., Du, X., Wang, Y., Peng, X.: Codegen4libs: a two-stage approach for library-oriented code generation. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 434–445. IEEE (2023)
23. Ma, L., et al.: Knowlog: knowledge enhanced pre-trained language model for log understanding. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, pp. 1–13 (2024)
24. Min, S., et al.: Rethinking the role of demonstrations: What makes in-context learning work? In: Goldberg, Y., Kozareva, Z., Zhang, Y. (eds.) Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, pp. 11048–11064. Association for Computational Linguistics, Abu Dhabi, United Arab Emirates (2022). https://doi.org/10.18653/v1/2022.emnlp-main.759. https://aclanthology.org/2022.emnlp-main.759
25. Pan, L., Albalak, A., Wang, X., Wang, W.: Logic-LM: empowering large language models with symbolic solvers for faithful logical reasoning. In: Findings of the Association for Computational Linguistics: EMNLP 2023, pp. 3806–3824 (2023)
26. Pérez, J., Barceló, P., Marinkovic, J.: Attention is turing-complete. J. Mach. Learn. Res. **22**(75), 1–35 (2021)
27. Roziere, B., et al.: Code llama: open foundation models for code. arXiv preprint arXiv:2308.12950 (2023)
28. Saparov, A., He, H.: Language models are greedy reasoners: a systematic formal analysis of chain-of-thought. In: The Eleventh International Conference on Learning Representations (2023). https://openreview.net/forum?id=qFVVBzXxR2V
29. Schuurmans, D.: Memory augmented large language models are computationally universal. arXiv preprint arXiv:2301.04589 (2023)
30. Shi, Z., et al.: Satformer: transformer-based unsat core learning. In: 2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD), pp. 1–4. IEEE (2023)
31. Soltana, G., Sabetzadeh, M., Briand, L.C.: Practical constraint solving for generating system test data. ACM Trans. Softw. Eng. Methodol. (TOSEM) **29**(2), 1–48 (2020)
32. Sun, M., Yang, Y., Wang, Y., Wen, M., Jia, H., Zhou, Y.: SMT solver validation empowered by large pre-trained language models. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 1288–1300. IEEE (2023)
33. Sun, Y., et al.: GPTScan: detecting logic vulnerabilities in smart contracts by combining GPT with program analysis. In: Proceedings of the IEEE/ACM ICSE (2024)
34. Tafjord, O., Dalvi, B., Clark, P.: Proofwriter: generating implications, proofs, and abductive statements over natural language. In: Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021, pp. 3621–3634 (2021)

35. Touvron, H., S., et al.: Llama 2: open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288 (2023)
36. Wang, C., Lou, Y., Liu, J., Peng, X.: Generating variable explanations via zero-shot prompt learning. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 748–760. IEEE (2023)
37. Wang, L., et al.: Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 2609–2634 (2023)
38. Wang, X., et al.: Self-consistency improves chain of thought reasoning in language models. In: The Eleventh International Conference on Learning Representations (2022)
39. Wei, C., Chen, Y., Ma, T.: Statistically meaningful approximation: a case study on approximating turing machines with transformers. In: Advances in Neural Information Processing Systems, vol. 35, pp. 12071–12083 (2022)
40. Wei, J., et al.: Chain-of-thought prompting elicits reasoning in large language models. In: Advances in Neural Information Processing Systems, vol. 35, pp. 24824–24837 (2022)
41. Xu, J., et al.: Unilog: automatic logging via LLM and in-context learning. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, pp. 1–12 (2024)
42. Yang, A.Z., Le Goues, C., Martins, R., Hellendoorn, V.: Large language models for test-free fault localization. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, pp. 1–12 (2024)
43. Ye, X., Chen, Q., Dillig, I., Durrett, G.: SatLM: satisfiability-aided language models using declarative prompting. In: Advances in Neural Information Processing Systems, vol. 36 (2024)
44. Yu, H., et al.: Codereval: a benchmark of pragmatic code generation with generative pre-trained models. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, pp. 1–12 (2024)
45. Zhang, S., Gu, X., Chen, Y., Shen, B.: Infere: step-by-step regex generation via chain of inference. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 1505–1515. IEEE (2023)
46. Zhang, Y., Zhen, H.L., Pei, Z., Lian, Y., Yin, L., Yuan, M., Yu, B.: Sola: solver-layer adaption of LLM for better logic reasoning. arXiv preprint arXiv:2402.11903 (2024)
47. Zhong, W., et al.: Analytical reasoning of text. In: Findings of the Association for Computational Linguistics: NAACL 2022, pp. 2306–2319 (2022)